hostrada4py - User Guide

Language: English

Documentation snapshot: 8 August 2026

Target: `hostrada4py` 0.42.0, `main` branch of the official GitHub repository

This user guide explains practical use of the Python library `hostrada4py`. It covers access to DWD HOSTRADA data and the optional CERRA provider, point, area and route analysis, decomposition of global solar radiation, and weather-file exports for simulation software.

Note

The library and the underlying data sources continue to evolve. For reproducible work, record the `hostrada4py` version, provider, time basis and cache strategy used in your project. This guide describes the repository state on 8 August 2026.

Contents

1. Purpose and scope

`hostrada4py` is an open-source Python library developed at Berlin University of the Arts, Chair of Building Services Engineering (VPT). It simplifies access to high-resolution weather data and provides analysis helpers that integrate directly with `pandas`, `xarray` and `GeoPandas` workflows.

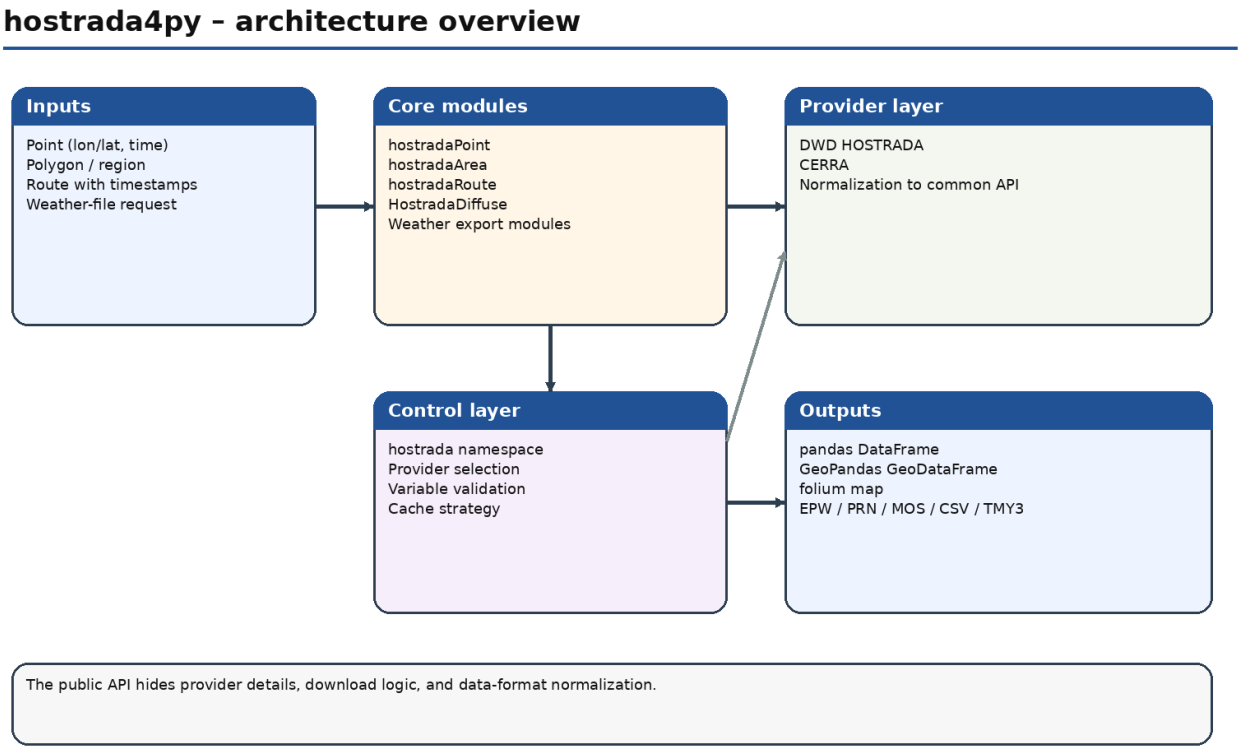
Typical use cases include:

- hourly weather time series for a location,
- simultaneous retrieval of several meteorological variables,
- derivation of diffuse and direct solar radiation from global radiation,
- spatial analysis over city, regional or custom polygons,
- temporal means and spatial summaries,
- interactive maps using `folium`,
- weather values along time-stamped routes,
- weather-file generation for EnergyPlus, IDA ICE, Polysun, SimStadt and Modelica BuildingSystems,
- switching between DWD HOSTRADA and Copernicus CERRA through a shared provider interface.

The public helpers use canonical variable codes. As a result, much of your application code can remain independent of the selected data source.

1.1 Architecture overview

The following figure shows the main building blocks of the library and how they work together at a high level.



Architecture overview of `hostrada4py`: typical inputs, core modules, control layer, providers, and major output formats.

For everyday use, the key idea is that the core modules rely on a shared provider interface. This means many calls remain almost unchanged even when you switch between DWD HOSTRADA and CERRA.

2. Installation

2.1 Requirements

`hostrada4py` 0.42.0 requires Python 3.10 or later. Most workflows also require an internet connection because monthly source files or CERRA raw data are downloaded on demand.

A virtual environment is recommended:

```
python -m venv .venv

# Linux/macOS
source .venv/bin/activate
```

```
# Windows PowerShell
.venv\Scripts\Activate.ps1
```

2.2 Install from the GitHub repository

The project README documents an editable installation from a clone of the repository:

```
git clone https://github.com/UdK-VPT/hostrada4py.git
cd hostrada4py
python -m pip install --upgrade pip
python -m pip install -e .
```

Verify the import and version:

```
import hostrada4py

print(hostrada4py.__version__)
# Expected for this guide snapshot: 0.42.0
```

Regular dependencies include `numpy`, `pandas`, `xarray`, `netcdf4`, `pyproj`, `shapely`, `geopandas`, `folium`, `matplotlib`, `plotly`, `pvlib`, `fsspec` and `h5netcdf`.

2.3 Optional CERRA dependencies

The Copernicus CERRA provider requires additional packages. With the current package definition they can be installed as an optional extra:

```
python -m pip install -e ".[cerra]"
```

Alternatively, use the repository's `requirements-cerra.txt`:

```
python -m pip install -r requirements-cerra.txt
```

The optional stack includes `cdsapi`, `cfgrib`, `eccodes` and `scipy`. You also need valid credentials for the Copernicus Climate Data Store. See section 12.

3. Data sources, variables and coordinates

3.1 DWD HOSTRADA

HOSTRADA is a high-resolution hourly gridded data set from the German Weather Service (DWD) for Germany. It covers German land areas at approximately 1 km x 1 km spatial resolution and extends back to 1995. Monthly files are distributed as NetCDF in the EPSG:3034 coordinate reference system. Time in the source data is UTC.

3.2 Copernicus CERRA

CERRA is a European regional reanalysis. The `hostrada4py` provider normalises CERRA data to a common interface and a rectilinear EPSG:3034 target grid. The current implementation uses a target grid spacing of about 5.5 km by default. Analysis fields are natively available every three hours and are normalised to an hourly interface; hourly forecast accumulations are used for global solar radiation. The provider supports data from September 1984. CERRA does not provide a field equivalent to HOSTRADA's urban heat island intensity.

3.3 Canonical variable codes

`hostrada4py` uses the following canonical codes. Always inspect the `unit` column or provider metadata in actual results before applying numerical thresholds or formulas because source formats can use different units.

`tas`

: Air temperature at 2 m.

`tdew`

: Dew-point temperature at 2 m.

`uhi`

: Urban heat island intensity. Available from DWD HOSTRADA only.

`sfcWind`

: Wind speed at 10 m.

`sfcWind_direction`

: Wind direction at 10 m.

`rsds`

Surface downwelling shortwave global radiation.

`clt`

Total cloud cover.

`hurs`

Relative humidity at 2 m.

`mixr`

Water-vapour mixing ratio at 2 m.

`ps`

Surface or station-level air pressure.

`psl`

Mean sea-level pressure.

Query the capabilities of the active provider programmatically:

```python
from hostrada4py import hostrada as hs

print(hs.available_providers())
print(hs.supported_variables())
print(hs.provider_capabilities())
```

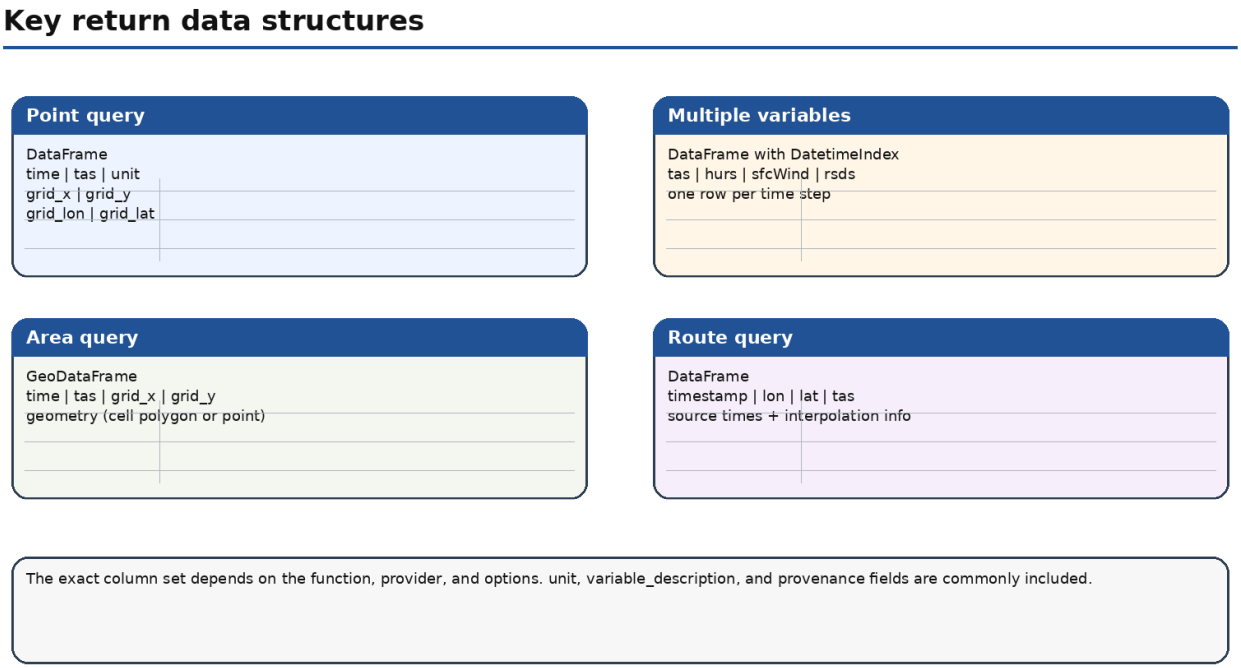
3.4 Coordinates

Public point, area and weather-file functions accept geographic longitude and latitude, typically `lon`/`lat` in EPSG:4326. Internally the library transforms these coordinates to the EPSG:3034 grid used by normalised provider files.

For a point query, the nearest grid cell is selected. Depending on the provider, the returned table includes input coordinates and metadata such as `grid_x_epsg3034`, `grid_y_epsg3034`, `grid_lon` and `grid_lat`. Use these fields to document which model/grid cell was actually evaluated.

3.5 Return data structures at a glance

The return types follow a clear pattern: point and route queries return tabular `pandas.DataFrame` objects, while area queries add a geometry column in a `GeoDataFrame`.



Simplified overview of the most important return data structures used by `hostrada4py`.

The exact column set depends on the function and provider. Still, time columns, variable values, units, grid metadata, and — depending on the workflow — additional provenance or geometry information are common recurring elements.

4. Core concepts: time, providers and caching

4.1 Time in point and area queries

Point and area extraction operates internally in UTC. For the point helpers, a timestamp without timezone information is interpreted as UTC. A timezone-aware value is converted to UTC.

For unambiguous and reproducible code, use explicit UTC timestamps:

```
start = "2024-07-01T00:00:00Z"
end = "2024-07-02T23:00:00Z"
```

4.2 Time on routes

The route API is designed around local travel times. Its default timezone is `Europe/Berlin`. Naive route timestamps are localised to the configured `timezone`; timezone-aware timestamps are converted to it. Ambiguous or non-existent local times at

daylight-saving transitions deliberately raise an error so that the route is not silently misinterpreted.

4.3 Providers

The default provider is `dwd`. A provider can be selected per function call, for a context block, or process-wide. For reproducible scripts, an explicit `provider=` argument is particularly clear.

```
from hostrada4py import hostrada as hs
from hostrada4py.hostradaPoint import extract_values_for_point

# Explicit for one call
df = extract_values_for_point(
    "tas", 13.405, 52.52,
    "2024-01-01T00:00:00Z", "2024-01-01T23:00:00Z",
    provider="dwd",
)

# Temporary context for several calls
with hs.use_provider("cerra"):
    print(hs.supported_variables())

# Process-wide default
hs.set_default_provider("dwd")
```

The process default can also be selected with the `HOSTRADA_PROVIDER` environment variable.

4.4 Cache

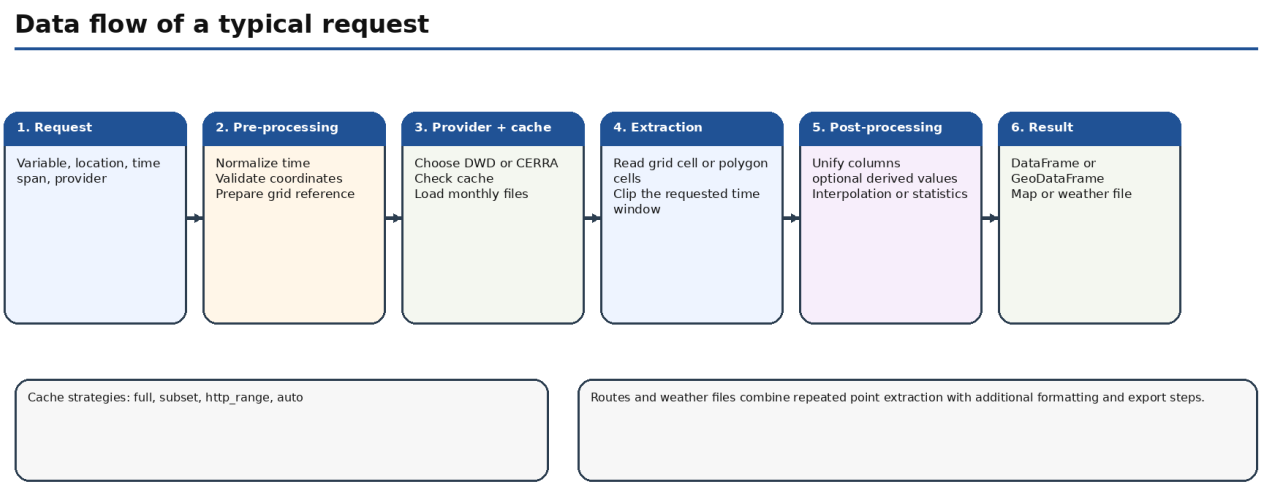
The default cache directory is `hostrada_cache`. DWD HOSTRADA files are cached by month. Repeated queries for an already cached month are therefore much faster and do not require another full download.

Use a project-specific cache if desired:

```
df = extract_values_for_point(
    "tas", 13.405, 52.52,
    "2024-01-01", "2024-01-02",
    cache_dir="./cache/hostrada",
)
```

4.5 Data flow of a typical request

Internally, many functions follow a similar processing pattern. The figure below summarizes the usual path from a user request to the final result.



Typical data flow in `hostrada4py`: from request validation via provider and cache logic to a normalized result.

For more advanced workflows — such as route processing, radiation decomposition, or weather-file export — several of these steps are combined or repeated. The core principle remains the same: validate inputs, obtain source data, normalize them into a shared structure, and return a usable result.

5. Quick start: weather data for a point ----------

The central point helper is `extract_values_for_point`.

```
from hostrada4py.hostradaPoint import extract_values_for_point

df = extract_values_for_point(
    var="tas",
    lon=13.4050,
    lat=52.5200,
    start="2024-07-01T00:00:00Z",
    end="2024-07-02T23:00:00Z",
    provider="dwd",
    cache_strategy="full",
    verbose=True,
)

print(df.head())
print(df.columns)
```

The result is a pandas.DataFrame. At minimum it contains time and the requested variable, tas in this example. Grid-cell metadata, unit and variable_description are normally added when available from the source file.

Important points:

- end must not be earlier than start.
- Input coordinates must be valid longitude/latitude values.
- Outside a provider's intended spatial coverage, the nearest cell may be scientifically inappropriate or the provider may reject the request.
- Inspect unit before using physical formulas or thresholds.
- For analyses spanning many months, keeping the cache can save substantial time.

A simple plot:

```
import matplotlib.pyplot as plt

ax = df.plot(x="time", y="tas", figsize=(10, 4), legend=False)
ax.set_title("Air temperature - Berlin")
ax.set_xlabel("Time")
ax.set_ylabel(df["unit"].dropna().iloc[0] if "unit" in df else "tas")
plt.tight_layout()
plt.show()
```

6. Multiple variables at one point

Use extract_multiple_values_for_point to retrieve several variables. Individual series are joined on the time axis into a wide DataFrame.

```
from hostrada4py.hostradaPoint import extract_multiple_values_for_point

weather = extract_multiple_values_for_point(
    variables=["tas", "hurs", "sfcWind", "rsds"],
    lon=13.4050,
    lat=52.5200,
    start="2024-07-01T00:00:00Z",
    end="2024-07-07T23:00:00Z",
    provider="dwd",
    cache_strategy="full",
)

print(weather.head())
print(weather.index)
```

The combined result uses a DatetimeIndex and has one column per weather variable. Location metadata from the first suitable single-variable result is kept.

A practical CSV export:

```
weather.reset_index().to_csv(
    "berlin_weather.csv",
    index=False,
    encoding="utf-8",
)
```

7. Diffuse and direct solar radiation

HOSTRADA and CERRA contain global radiation but not diffuse horizontal irradiance (DHI) directly. `hostrada4py` can decompose global radiation using `pvlib` and the `erbs_driesse` model.

7.1 Convenience function

```
from hostrada4py import extract_diffuse_radiation_for_point

solar = extract_diffuse_radiation_for_point(
    lon=13.405,
    lat=52.52,
    start="2024-06-01T00:00:00Z",
    end="2024-06-03T23:00:00Z",
    tz="Europe/Berlin",
    provider="dwd",
    apply_weather_correction=False,
)

print(solar[["dhi", "dni", "kd"]].head())
```

Key derived quantities are:

`dhi`

: Diffuse horizontal irradiance.

`dni`

: Direct normal irradiance.

`kd`

: Diffuse fraction of global irradiance, bounded between 0 and 1.

Solar-position columns such as `solar_zenith_deg`, `solar_elevation_deg` and `solar_azimuth_deg` are also added.

7.2 Optional weather correction

With `apply_weather_correction=True`, the robust `erbs_driesse` base estimate is adjusted by a bounded, conservative weather correction. Depending on provider availability, additional variables such as cloud cover, relative humidity or UHI can be used. Missing optional correction variables are not forced.

```
solar_corrected = extract_diffuse_radiation_for_point(
    lon=13.405,
    lat=52.52,
    start="2024-06-01T00:00:00Z",
    end="2024-06-03T23:00:00Z",
    tz="Europe/Berlin",
    apply_weather_correction=True,
)
```

For scientific or standards-based work, document whether the optional correction was enabled and validate the derived radiation quantities against the requirements of your downstream model.

7.3 Using HostradaDiffuse directly

If you already have a DataFrame containing `rsds`, use `HostradaDiffuse` directly. The input index must be a `pandas.DatetimeIndex`.

```
from hostrada4py import HostradaDiffuse

model = HostradaDiffuse(
    latitude=52.52,
    longitude=13.405,
    altitude=34.0,
    tz="Europe/Berlin",
)

result = model.estimate(weather, apply_weather_correction=False)
```

8. Area analysis with polygons

`hostradaArea` extracts all selected grid cells for a polygon. Polygons are passed as sequences of `(lon, lat)` coordinate pairs.

8.1 Custom polygon

```python
from hostrada4py.hostradaArea import extract_values_for_polygon

polygon = [
    (13.30, 52.45),
    (13.55, 52.45),
    (13.55, 52.60),
    (13.30, 52.60),
    (13.30, 52.45),
]

area = extract_values_for_polygon(
    var="tas",
    polygon_lonlat=polygon,
    start_utc="2024-07-01T12:00:00Z",
    end_utc="2024-07-01T15:00:00Z",
    selection_mode="within",
    return_geodataframe=True,
    provider="dwd",
    cache_strategy="subset",
    subset_margin_cells=1,
)

print(area.head())
print(area.crs)
```

With `return_geodataframe=True` the result is a `GeoDataFrame` in EPSG:3034. Typical columns are `time`, `grid_y_epsg3034`, `grid_x_epsg3034`, the value column, `geometry`, `selection_mode`, `grid_lon` and `grid_lat`.

8.2 Predefined cities

`hostradaCities` contains rough bounding polygons for the 50 largest German cities plus Speyer and Korbach. These are intentionally simple selection envelopes, not official administrative boundaries.

```python
from hostrada4py.hostradaCities import CITY_POLYGONS
from hostrada4py.hostradaArea import extract_values_for_polygon

berlin = CITY_POLYGONS["Berlin"]

berlin_tas = extract_values_for_polygon(
    "tas",
    berlin,
    "2024-07-01T12:00:00Z",
```

```
    "2024-07-01T12:00:00Z",
)
```

Compatibility variables such as `berlin_polygon` are also provided.

8.3 Predefined regions

`hostradaRegions` contains simple selection polygons for Germany and several states or regions.

```
from hostrada4py.hostradaRegions import REGIONS_POLYGONS

germany = REGIONS_POLYGONS["Germany"]
brandenburg = REGIONS_POLYGONS["Brandenburg"]
```

These geometries are selection boxes as well and should not be interpreted as official boundary data.

8.4 Temporal mean per grid cell

`extract_mean_values_for_polygon` reduces a period to one row per grid cell and can include additional statistics.

```
from hostrada4py.hostradaArea import extract_mean_values_for_polygon

mean_area = extract_mean_values_for_polygon(
    var="tas",
    polygon_lonlat=berlin,
    start_utc="2024-07-01T00:00:00Z",
    end_utc="2024-07-07T23:00:00Z",
    include_statistics=True,
)

print(mean_area.head())
```

8.5 Spatial summary per timestamp

If you already have the full area time series, `summarize_values_period` computes mean, minimum, maximum and selected cell count for each timestamp.

```
from hostrada4py.hostradaArea import summarize_values_period
```

```
summary = summarize_values_period(area, "tas")
print(summary.head())
```

9. Mapping area data

hostradaArea can create folium maps for an individual timestamp or for a time-aggregated data set.

9.1 Map of one timestamp

```
from hostrada4py.hostradaArea import make_leaflet_map_timepoint

m = make_leaflet_map_timepoint(
    area,
    var="tas",
    time_utc="2024-07-01T12:00:00Z",
    show_cell_values=True,
    decimals=1,
    title="Berlin - air temperature",
    save_html="berlin_tas_2024-07-01_12UTC.html",
)
```

In Jupyter, the returned folium.Map object is displayed directly. In a script, save_html is useful for opening the interactive map in a browser.

9.2 Map of an aggregated period

```
from hostrada4py.hostradaArea import make_leaflet_map_timeperiod

m_mean = make_leaflet_map_timeperiod(
    mean_area,
    var="tas",
    show_cell_values=False,
    title="Berlin - mean air temperature",
    save_html="berlin_tas_mean.html",
)
```

Use vmin and vmax to keep a consistent colour scale across comparable maps. reverse_colormap=True reverses the colour scale.

10. Climate data along a route

The route API combines the position and timestamp of each route point with weather data. It accepts either a `pandas.DataFrame` or a CSV file.

10.1 Input columns

The following names are recognised case-insensitively:

- time: `timestamp`, `time`, `datetime` or `date_time`
- longitude: `longitude`, `lon`, `lng` or `x`
- latitude: `latitude`, `lat` or `y`

10.2 DataFrame example

```
import pandas as pd
from hostrada4py.hostradaRoute import calculate_route_climate

route = pd.DataFrame(
    {
        "timestamp": [
            "2024-07-01 08:00:00",
            "2024-07-01 08:30:00",
            "2024-07-01 09:00:00",
        ],
        "longitude": [13.4050, 13.20, 12.95],
        "latitude": [52.5200, 52.48, 52.40],
    }
)

route_weather = calculate_route_climate(
    route,
    variables=["tas", "hurs", "sfcWind", "rsds"],
    timezone="Europe/Berlin",
    interpolation="linear",
    provider="dwd",
    output_csv="route_weather.csv",
)

print(route_weather.head())
```

The default interpolation mode is `linear`; `nearest` is also available. Linear interpolation of wind direction handles the cyclic transition at 0/360 degrees.

For every variable the output includes not only the converted display value but also provenance fields such as the raw value, source unit, two source UTC times, interpolation fraction and selected grid coordinates.

10.3 All supported variables

`variables="all"` selects all variables known to the route module and supported by the active provider.

```
from hostrada4py.hostradaRoute import available_variables

print(available_variables(provider="dwd"))
```

10.4 Command line

The route module includes a small command-line entry point:

```
python -m hostrada4py.hostradaRoute route.csv \
    -o route_weather.csv \
    -v tas hurs rsds \
    --timezone Europe/Berlin \
    --interpolation linear \
    --cache-strategy full \
    --provider dwd
```

For routes, `full` is a sensible default because many points in one month can reuse the same monthly files.

11. Weather files for simulation programs

The repository includes exporters for several simulation environments. They follow the same general pattern: provide location and period, optionally choose provider, timezone, altitude, cache strategy and radiation correction, then write the target file.

11.1 EnergyPlus EPW

```
from hostrada4py.hostrada_EnergyPlus_Weather import (
    create_energyplus_weather_file,
)

path = create_energyplus_weather_file(
```

```
    lon=13.405,
    lat=52.52,
    start="2024-01-01T00:00:00Z",
    end="2024-12-31T23:00:00Z",
    output_file="Berlin_2024.epw",
    location_name="Berlin",
    country="DEU",
    tz="Europe/Berlin",
    altitude=34.0,
    provider="dwd",
    cache_strategy="subset",
    subset_margin_cells=1,
)

print(path)
```

11.2 IDA ICE

```
from hostrada4py.hostrada_IDA_ICE_Weather import (
    create_ida_ice_weather_file,
)

create_ida_ice_weather_file(
    13.405, 52.52,
    "2024-01-01T00:00:00Z",
    "2024-12-31T23:00:00Z",
    output_file="Berlin_2024.prn",
    tz="Europe/Berlin",
    provider="dwd",
)
```

11.3 Polysun

```
from hostrada4py.hostrada_Polysun_Weather import (
    create_polysun_weather_file,
)

create_polysun_weather_file(
    13.405, 52.52,
    "2024-01-01T00:00:00Z",
    "2024-12-31T23:00:00Z",
    output_file="Berlin_2024_Polysun.csv",
    tz="Europe/Berlin",
)
```

11.4 SimStadt

```
from hostrada4py.hostrada_SimStadt_Weather import (
    create_simstadt_weather_file,
)

create_simstadt_weather_file(
    13.405, 52.52,
    "2024-01-01T00:00:00Z",
    "2024-12-31T23:00:00Z",
    output_file="Berlin_2024.tmy3",
    tz="Europe/Berlin",
)
```

11.5 Modelica BuildingSystems

```
from hostrada4py.hostrada_BuildingSystems_Weather import (
    create_buildingsystems_weather_file,
)

create_buildingsystems_weather_file(
    13.405, 52.52,
    "2024-01-01T00:00:00Z",
    "2024-12-31T23:00:00Z",
    output_file="Berlin_2024.mos",
    tz="UTC",
)
```

Warning

Before running a simulation, verify that period, timezone, leap-year handling, altitude, radiation quantities and units meet the requirements of the target program. A successfully generated file format is not by itself a scientific validation of the weather series.

12. Switching providers: DWD HOSTRADA and CERRA

12.1 Providers and capabilities

```
from hostrada4py import hostrada as hs
```

```
for name in hs.available_providers():
    cap = hs.provider_capabilities(name)
    print(name)
    print("  Variables:", sorted(cap.variables))
    print("  Time resolution:", cap.temporal_resolution)
    print("  Grid [m]:", cap.spatial_resolution_m)
    print("  CRS:", cap.crs)
    print("  Start:", cap.start)
```

The current registry contains `dwd` and `cerra`. DWD also has the aliases `hostrada` and `dwd_hostrada`; CERRA has `copernicus_cerra`.

12.2 Configure CERRA access

CERRA requires a Copernicus Climate Data Store account and API credentials. The library checks an optional file `hostrada4py/cerra_config.json`. If it is missing or contains no relevant values, `cdsapi` falls back to its standard configuration, such as `~/.cdsapirc` and supported environment variables.

A possible local JSON structure is:

```
{
  "url": "<CDS-API-URL>",
  "key": "<YOUR-API-KEY>"
}
```

Do not commit real credentials to a public Git repository.

12.3 Use CERRA

```
from hostrada4py.hostradaPoint import extract_values_for_point

cerra = extract_values_for_point(
    "tas",
    lon=2.3522,
    lat=48.8566,
    start="2024-01-01T00:00:00Z",
    end="2024-01-02T23:00:00Z",
    provider="cerra",
)

print(cerra.head())
```

Key differences compared with DWD HOSTRADA:

- CERRA covers Europe rather than Germany only.

- Its normalised target grid is coarser than HOSTRADA.
- The current provider starts in September 1984.
- UHI is not available.
- Most native CERRA analysis fields are three-hourly; the provider layer exposes a normalised hourly interface.
- CDS retrieval can involve large raw files. The current CERRA retrieval process does not expose true geographic subsetting for this data set, so the provider crops locally after retrieval.

13. Cache and download strategies

The DWD HOSTRADA endpoint serves static monthly files. `hostrada4py` provides several strategies to balance transfer volume and local disk usage.

`full`

: Download and keep the complete monthly file. This is the robust default and is especially useful when many different points or areas from the same month are queried.

`subset`

: Use the full file when necessary, create a smaller spatial/temporal NetCDF subset and reuse that subset for later compatible requests.

`http_range`

: Best-effort HTTP byte-range/chunked access. Whether it offers a real advantage depends on server behaviour, file layout and installed optional packages.

`auto`

: Try `http_range` first, then fall back to `subset`.

Example:

```
df = extract_values_for_point(
    "tas",
    lon=13.405,
    lat=52.52,
    start="2024-01-01T00:00:00Z",
    end="2024-01-31T23:00:00Z",
    cache_strategy="subset",
    subset_margin_cells=1,
)
```

Important environment variables:

`HOSTRADA_NETCDF_SUBSET_MODE`

Process-wide default for the DWD cache strategy, for example `subset`.

`HOSTRADA_NETCDF_SUBSET_MARGIN_CELLS`

Extra margin in grid cells around a subset.

`HOSTRADA_NETCDF_SUBSET_FALLBACK`

Controls whether failed `http_range` access falls back to robust full-file retrieval. Set to `0` to disable fallback.

`HOSTRADA_DROP_FULL_AFTER_SUBSET`

Set to 1 to remove a full monthly file after a subset has been created, reducing local disk use.

Example in Python before the first request:

```
import os

os.environ["HOSTRADA_NETCDF_SUBSET_MODE"] = "subset"
os.environ["HOSTRADA_NETCDF_SUBSET_MARGIN_CELLS"] = "1"
```

Practical guidance:

- One site over many months: `subset` can reduce local storage.
- Many sites/areas in the same month: `full` is often more efficient.
- Routes with many points: `full` is the deliberate route-API default.
- Treat `http_range` as an optimisation, not as a guarantee of server-side NetCDF subsetting.

14. Post-processing and quality assurance

14.1 Inspect data before analysis

Before aggregating or exporting results, inspect them:

```
print(df.head())
print(df.tail())
print(df.dtypes)
print(df.isna().sum())
print(df["time"].min(), df["time"].max())
```

For point data, also record the actual grid cell and source unit.

14.2 Check the time axis

For local-time analysis, use an explicit conversion:

```
import pandas as pd

local = df.copy()
local["time"] = (
    pd.to_datetime(local["time"], utc=True)
    .dt.tz_convert("Europe/Berlin")
)
```

This keeps daylight-saving transitions explicit.

14.3 Store reproducibility metadata

For scientific or long-lived projects, store at least:

- hostrada4py version,
- provider name,
- variable code and returned unit,
- point coordinate or polygon,
- time period and timezone,
- cache strategy,
- for solar decomposition: method and weather-correction setting,
- date of retrieval, because the underlying data sets are extended monthly.

Example:

```
import hostrada4py
from hostrada4py import hostrada as hs

metadata = {
    "hostrada4py_version": hostrada4py.__version__,
    "provider": hs.get_provider_name(),
    "query_timezone": "UTC",
    "cache_strategy": "full",
}
print(metadata)
```

15. Troubleshooting

`ModuleNotFoundError: hostrada4py`

Confirm that the intended virtual environment is active and that `python -m pip install -e .` was run in the repository.

CERRA: `ImportError` for `cdsapi`, `cfgrib` or `eccodes`

Install the optional CERRA dependencies. `eccodes` may require additional native/system packages depending on the operating system.

CERRA: authentication or CDS errors

Check your account, accepted data-set terms and API configuration. Start with a short test request.

`Unknown weather provider`

Check `hs.available_providers()` and use a name such as `dwd` or `cerra`.

`Provider ... does not provide: uhi`

UHI is HOSTRADA-specific and is intentionally not provided by CERRA.

`end must not be earlier than start`

Correct the requested time bounds. Prefer ISO-8601 values with explicit Z for UTC.

Empty area result or no suitable cells

Check the order and location of `(lon, lat)` coordinates, provider coverage and selection geometry. Visualise the polygon separately if necessary.

Very large downloads or growing cache

Consider `subset` for long single-site series or deliberately remove cache directories that are no longer needed. `HOSTRADA_DROP_FULL_AFTER_SUBSET=1` can save disk space but can increase later download cost for different subsets.

Route fails at a daylight-saving transition

The route API deliberately raises for ambiguous or non-existent local times. Use unambiguous timezone-aware timestamps with UTC offsets, or set the route timezone to UTC.

DWD network errors

Retry later and retain complete monthly files already downloaded. If experimental `http_range` access is unstable, `auto` or `full` is generally more robust.

16. API quick reference

Point

```
extract_values_for_point(
    var, lon, lat, start, end,
    cache_dir="hostrada_cache",
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
    verbose=True,
) -> pandas.DataFrame

extract_multiple_values_for_point(
    variables, lon, lat, start, end,
    cache_dir="hostrada_cache",
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
    verbose=True,
) -> pandas.DataFrame
```

Historic aliases: `extract_values` and `extract_multiple_variables_for_point`.

Diffuse radiation

```
extract_diffuse_radiation_for_point(
    lon, lat, start, end,
    tz="UTC",
    apply_weather_correction=False,
    cache_dir="hostrada_cache",
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
    verbose=True,
) -> pandas.DataFrame
```

Area

```
extract_values_for_polygon(
    var,
    polygon_lonlat,
    start_utc,
```

```
    end_utc,
    cache_dir="hostrada_cache",
    selection_mode="within",
    return_geodataframe=True,
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
    verbose=True,
)

extract_mean_values_for_polygon(
    var,
    polygon_lonlat,
    start_utc,
    end_utc,
    cache_dir="hostrada_cache",
    selection_mode="within",
    return_geodataframe=True,
    mean_column=None,
    include_statistics=True,
    cache_strategy=None,
    subset_margin_cells=None,
    provider=None,
)

summarize_values_period(gdf_or_df, var) -> pandas.DataFrame
```

Maps

```
make_leaflet_map_timepoint(gdf_or_df, var, time_utc, ...,
save_html=None)
make_leaflet_map_timeperiod(gdf_or_df, var, ..., save_html=None)
```

Route

```
calculate_route_climate(
    route,
    variables="all",
    output_csv=None,
    timezone="Europe/Berlin",
    interpolation="linear",
    cache_dir="hostrada_cache",
    cache_strategy="full",
    subset_margin_cells=0,
    continue_on_error=False,
```

```
    progress_callback=None,
    provider=None,
)
```

Aliases: `calculate_hostrada_values_along_route` and `extract_values_for_route`.

Providers

```
hostrada.available_providers()
hostrada.get_provider(provider=None)
hostrada.get_provider_name(provider=None)
hostrada.supported_variables(provider=None)
hostrada.provider_capabilities(provider=None)
hostrada.set_default_provider(provider)
hostrada.use_provider(provider)
```

Weather files

```
create_energyplus_weather_file(...,
output_file="HOSTRADA_EnergyPlus.epw", ...)
create_ida_ice_weather_file(...,
output_file="HOSTRADA_IDA_ICE.prn", ...)
create_polysun_weather_file(...,
output_file="HOSTRADA_Polysun.csv", ...)
create_simstadt_weather_file(...,
output_file="HOSTRADA_SimStadt.tmy3", ...)
create_buildingsystems_weather_file(...,
output_file="HOSTRADA_BuildingSystems.mos", ...)
```

17. Notebooks and extended examples

The repository includes Jupyter notebooks illustrating the main workflows:

- `hostradaPoint.ipynb` - time series for one location
- `hostradaPoint-diffRad.ipynb` - diffuse radiation
- `hostradaRoute.ipynb` - weather values along a route
- `hostradaArea.ipynb` - two-dimensional weather fields
- `hostradaAreaMean.ipynb` - temporal means of spatial fields
- `hostradaHeatPeriods.ipynb` - intensive heat periods
- `hostradaHeatingDegreeDays.ipynb` - heating degree days
- `hostradaGenerateWeatherFiles.ipynb` - simulation weather files

The notebooks are useful for seeing complete analysis chains and visualisations. For production scripts, import the documented functions directly and make provider, time basis and caching choices explicit.

18. License and sources

The `hostrada4py` repository distributes the library under the MIT License. The meteorological source data remain subject to the terms and attribution requirements of DWD and Copernicus. Check those requirements before publishing or redistributing derived data sets.

Key sources for this guide:

- Official repository: https://github.com/UdK-VPT/hostrada4py
- README: https://github.com/UdK-VPT/hostrada4py/blob/main/README.md
- Package metadata: https://github.com/UdK-VPT/hostrada4py/blob/main/pyproject.toml
- DWD HOSTRADA Open Data: https://opendata.dwd.de/climate_environment/CDC/grids_germany/hourly/hostrada/
- Copernicus CERRA: https://climate.copernicus.eu/copernicus-regional-reanalysis-europe-cerra
- Copernicus Climate Data Store: https://cds.climate.copernicus.eu/

End of user guide.